



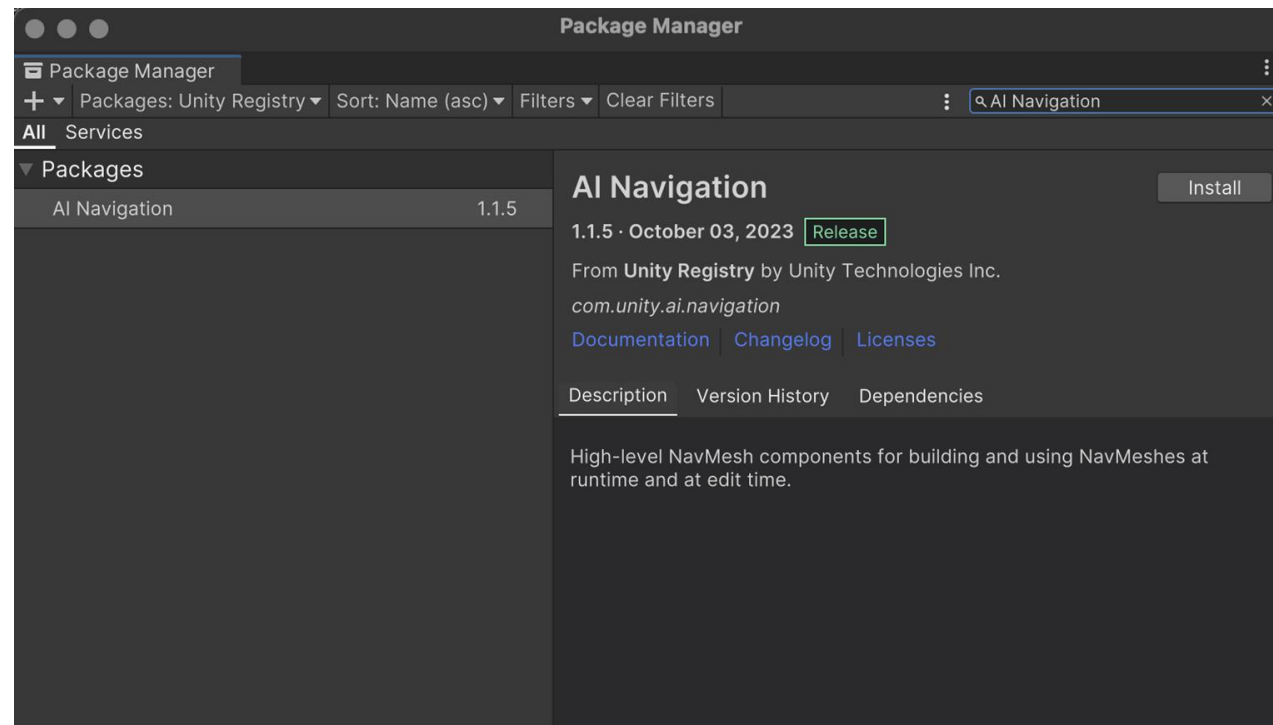
LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 4: NavMesh Navigation trong Unity

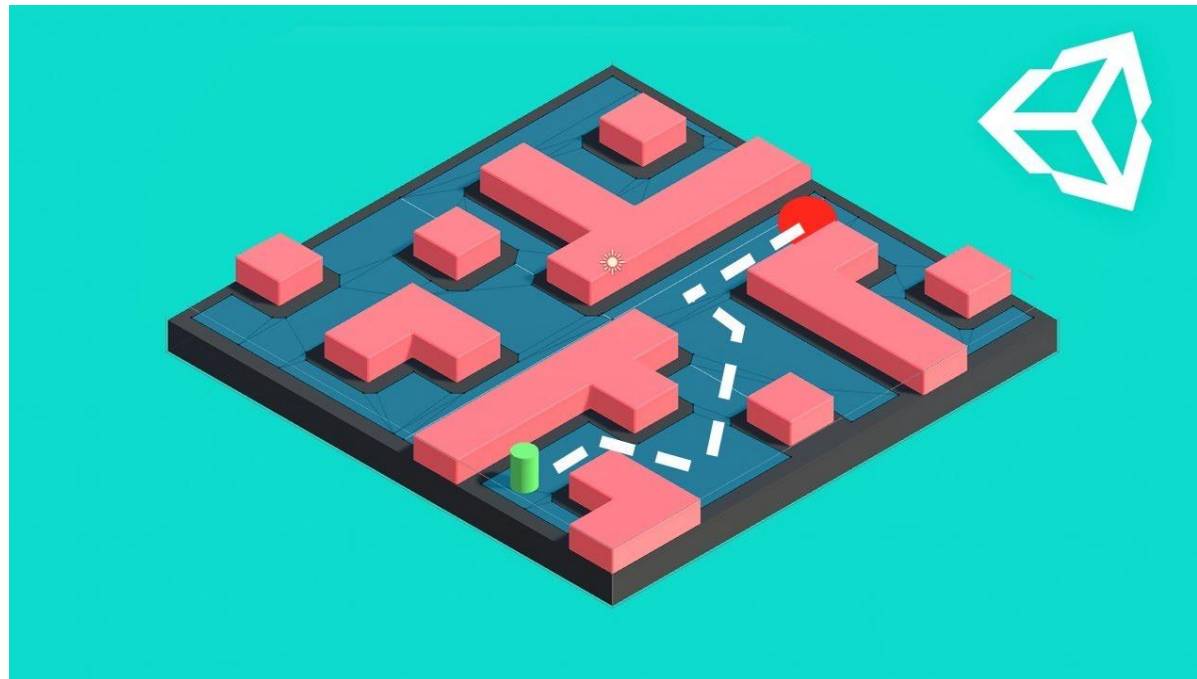
- ☐ AI Navigation
- ☐ Cách tính năng chính của AI Navigation

- ❑ AI : Artificial Intelligence - đề cập đến khả năng điều khiển các nhân vật trong game để chúng có thể di chuyển và phản ứng một cách thông minh với môi trường xung quanh, khái niệm AI này áp dụng trong bối cảnh phát triển game, không giống khái niệm AI như ChatGPT hay Gemini
- ❑ AI Navigation : tạo ra các nhân vật có thể di chuyển một cách thông minh trong thế giới game

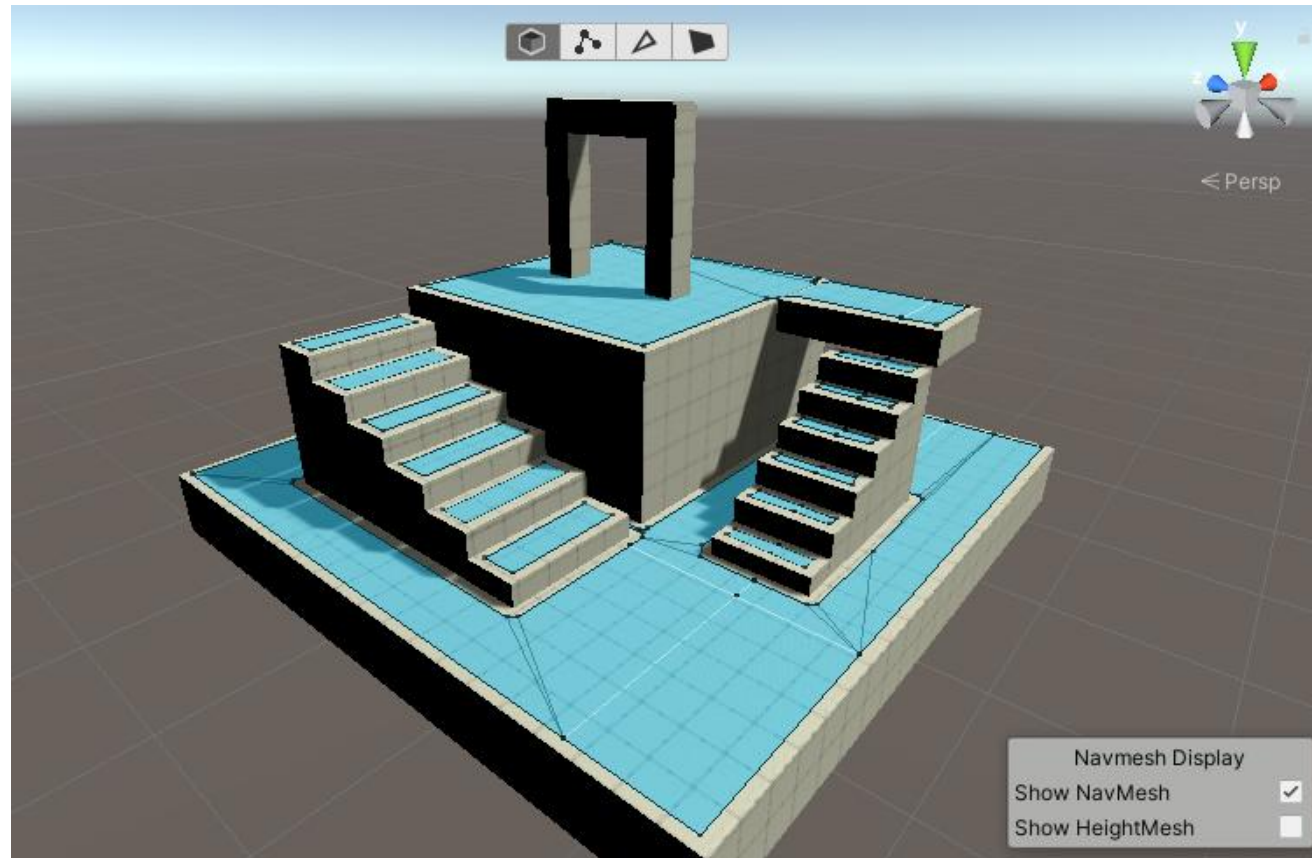
□ AI Navigation : Cần được cài đặt từ Package Manager



- Những nhân vật này sẽ dựa vào các lưới điều hướng (NavMesh) – được tự động tạo từ các vật thể trong cảnh – để xác định đường đi



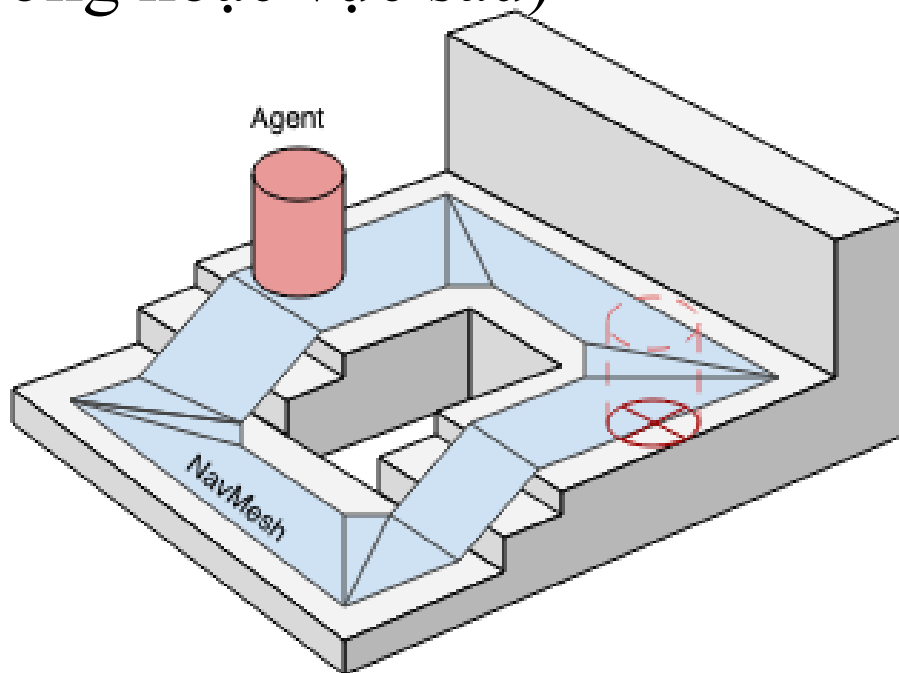
- Ví dụ về NavMesh (Lưới điều hướng) cho nhân vật di chuyển trong khu vực màu xanh



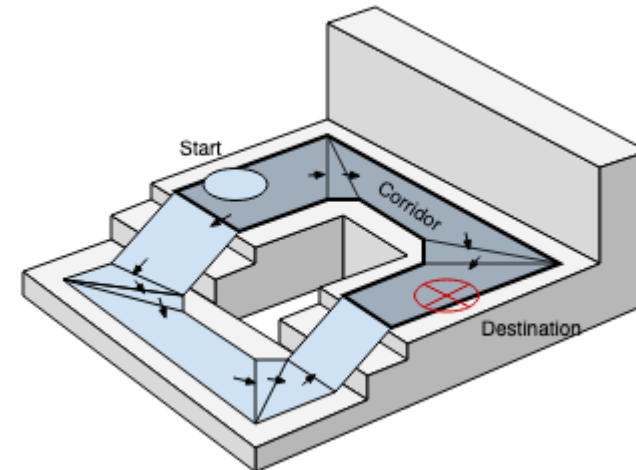
- ☐ Cách tính năng chính của AI Navigation
- ☐ Walkable Areas
- ☐ Finding Paths
- ☐ Following the Path
- ☐ Avoiding Obstacles
- ☐ Moving the Agent

- ☐ Global and Local
- ☐ Two Cases for Obstacles
- ☐ Shortcuts
- ☐ Obstacles
- ☐ NavMesh Links

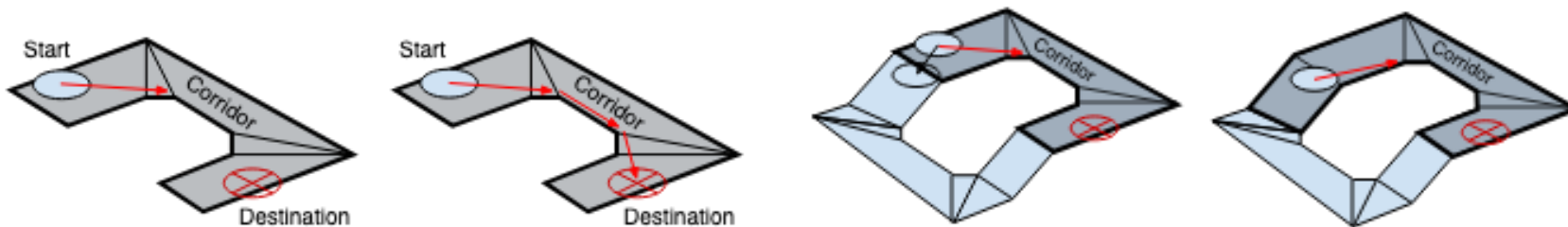
- **Walkable Areas** : giúp xác định các vùng mà nhân vật AI có thể di chuyển, đảm bảo rằng nhân vật không di chuyển qua các vùng không hợp lý (như tường hoặc vực sâu)



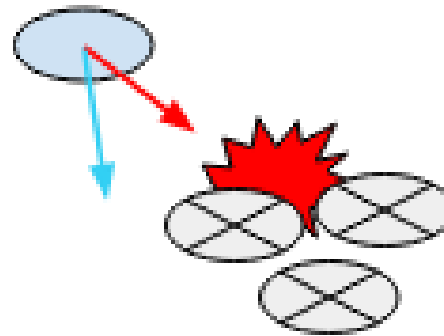
- **Finding Paths** : Sử dụng thuật toán A^* để tìm đường từ vị trí bắt đầu đến điểm đích bằng cách duyệt các đa giác gần nhất trong NavMesh.
- **A^*** : tìm ra con đường ngắn nhất từ điểm bắt đầu đến điểm đích, thông qua các ô hoặc nút trong một lưới hoặc đồ thị



❑ **Following the Path** : Nhân vật di chuyển theo lộ trình đã định (gọi là corridor), hướng về góc tiếp theo của corridor (lộ trình tính theo đa giác trong NavMesh) để tới đích.



- **Avoiding Obstacles** : Sử dụng thuật toán RVO (reciprocal velocity obstacles) để điều chỉnh hướng và vận tốc, giúp nhân vật tránh va chạm với chướng ngại vật và nhân vật khác.

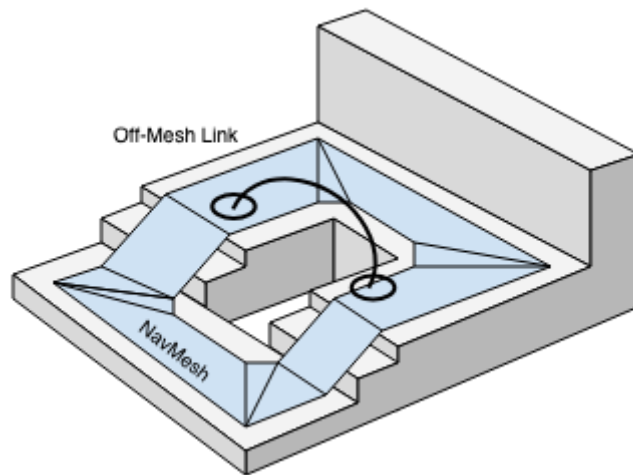


- ❑ **Moving the Agent** : Vận tốc cuối cùng của nhân vật được tính toán để mô phỏng chuyển động tự nhiên và mượt mà, có thể tích hợp vào hệ thống animation.
- ❑ **Global and Local Navigation** : Điều hướng toàn cục giúp tìm đường dài, trong khi điều hướng cục bộ điều chỉnh hướng đi ngắn hạn để tránh va chạm.

❑ Two Cases for Obstacles

- ❑ Static Obstacle: Đã tính vào NavMesh, agent không thể đi xuyên.
- ❑ Dynamic Obstacle: Xuất hiện, biến mất hoặc di chuyển trong runtime, agent tự tính lại đường đi nếu cần.

- ❑ **Voxels:** Quá trình tạo NavMesh sử dụng **voxelization** để xây dựng lưới điều hướng từ hình học Scene, giúp tăng độ chính xác của NavMesh.



- ❑ **Obstacles:** Xử lý chương ngại vật động bằng tránh va chạm cục bộ, còn chương ngại cố định sẽ tác động lên NavMesh bằng cách "carving" (cắt bỏ vùng).
- ❑ **NavMesh Links:** Kết nối các vùng NavMesh để nhân vật có thể thực hiện các hành động đặc biệt, như nhảy qua rào.

- ☐ AI Navigation
- ☐ Cách tính năng chính của AI Navigation
- ☐ Thực hành NavMesh Link



LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 4: FSM và Behavior Trees trong UNITY 3D

☐ Finite State Machine

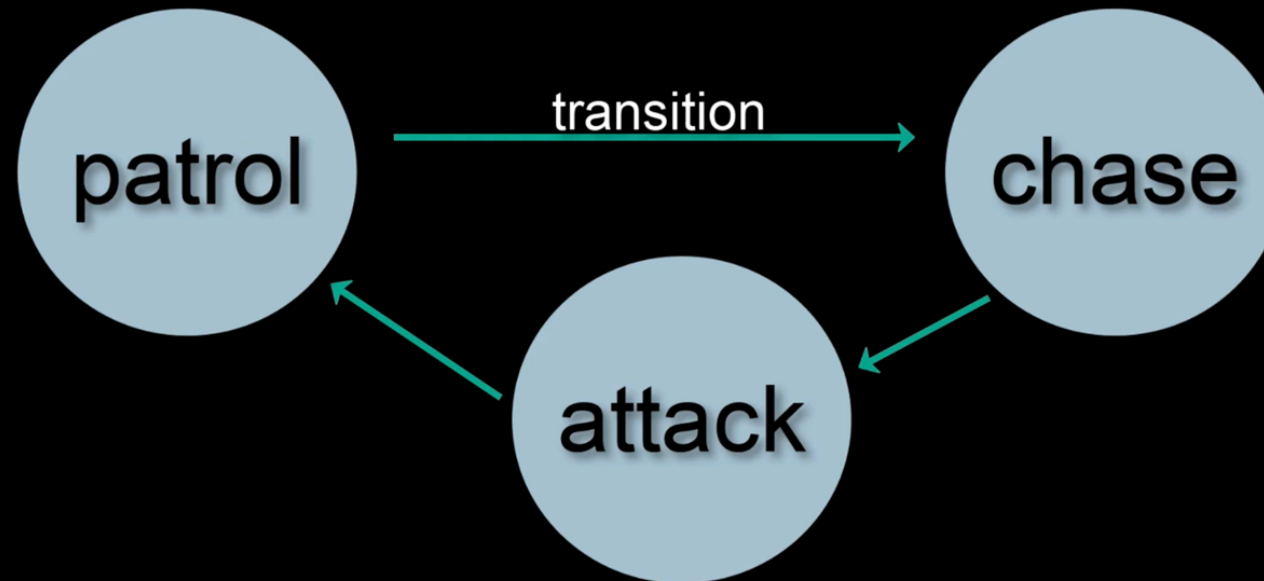
☐ Behavior Trees

□ **Finite State Machine (FSM)** : một mẫu thiết kế (design pattern) phổ biến, thường dùng khi viết câu lệnh điều khiển 1 NPC AI để quản lý các trạng thái khác nhau của một NPC. FSM cho phép đối tượng (NPC) có thể thực hiện các hành vi khác nhau tùy thuộc vào trạng thái hiện tại, như tuần tra, đuổi theo, hoặc tấn công.

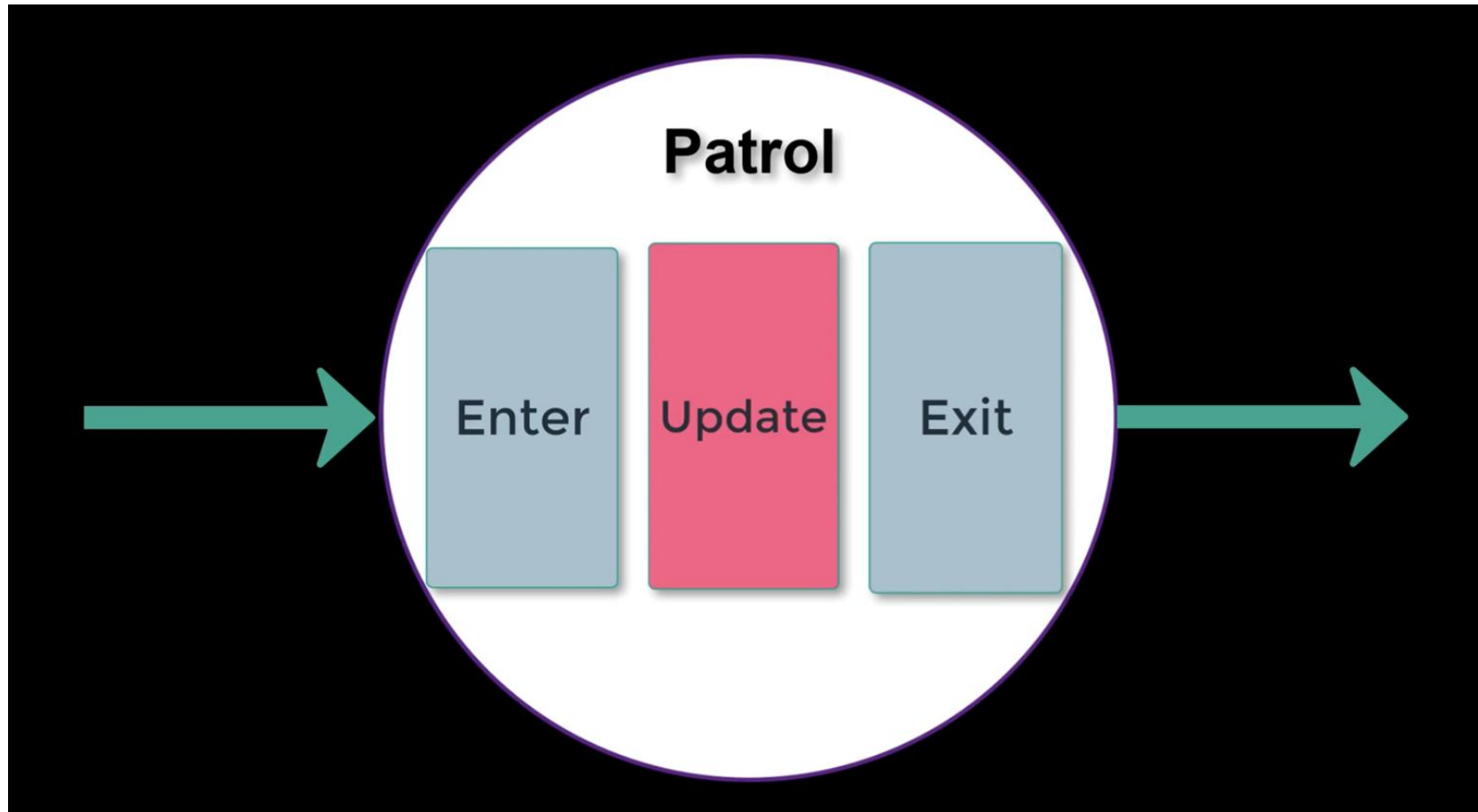
□ **NPC** : Non-player character

- ❑ **State** : Chia trạng thái cho NPC như Idle (đứng yên), Patrol (tuần tra), hoặc Chase (đuổi theo).
- ❑ **Transitions** : NPC sẽ chuyển sang trạng thái khác khi một điều kiện nhất định được đáp ứng, ví dụ, khi NPC phát hiện người chơi trong một khoảng cách nhất định.

Finite State Machine (FSM)



- ❑ Mỗi trạng thái có ba giai đoạn chính: Enter, Update, và Exit.
- ❑ **Enter:** Giai đoạn này bắt đầu khi NPC chuyển vào trạng thái mới, có thể thiết lập các hoạt động ban đầu.
- ❑ **Update:** Chạy liên tục khi NPC ở trong trạng thái đó.
- ❑ **Exit:** Xảy ra khi NPC thoát khỏi trạng thái.



- Định nghĩa 1 class điều khiển NPC theo cấu trúc FSM

```
8
9      // định nghĩa trạng thái cho nhân vật
      ? usages   ? inheritors   ? extension methods   ? exposing APIs
10     private enum State { Idle, Patrol, Chase }
11     private State currentState;
12 →
```

□ Định nghĩa 1 class điều khiển NPC theo cấu trúc

FSM

```
23 void Update()
24 {
25     // FSM xử lý các trạng thái
26     switch (currentState)
27     {
28         case State.Idle:
29             Idle();
30             break;
31         case State.Patrol:
32             Patrol();
33             break;
34         case State.Chase:
35             Chase();
36             break;
37     }
```

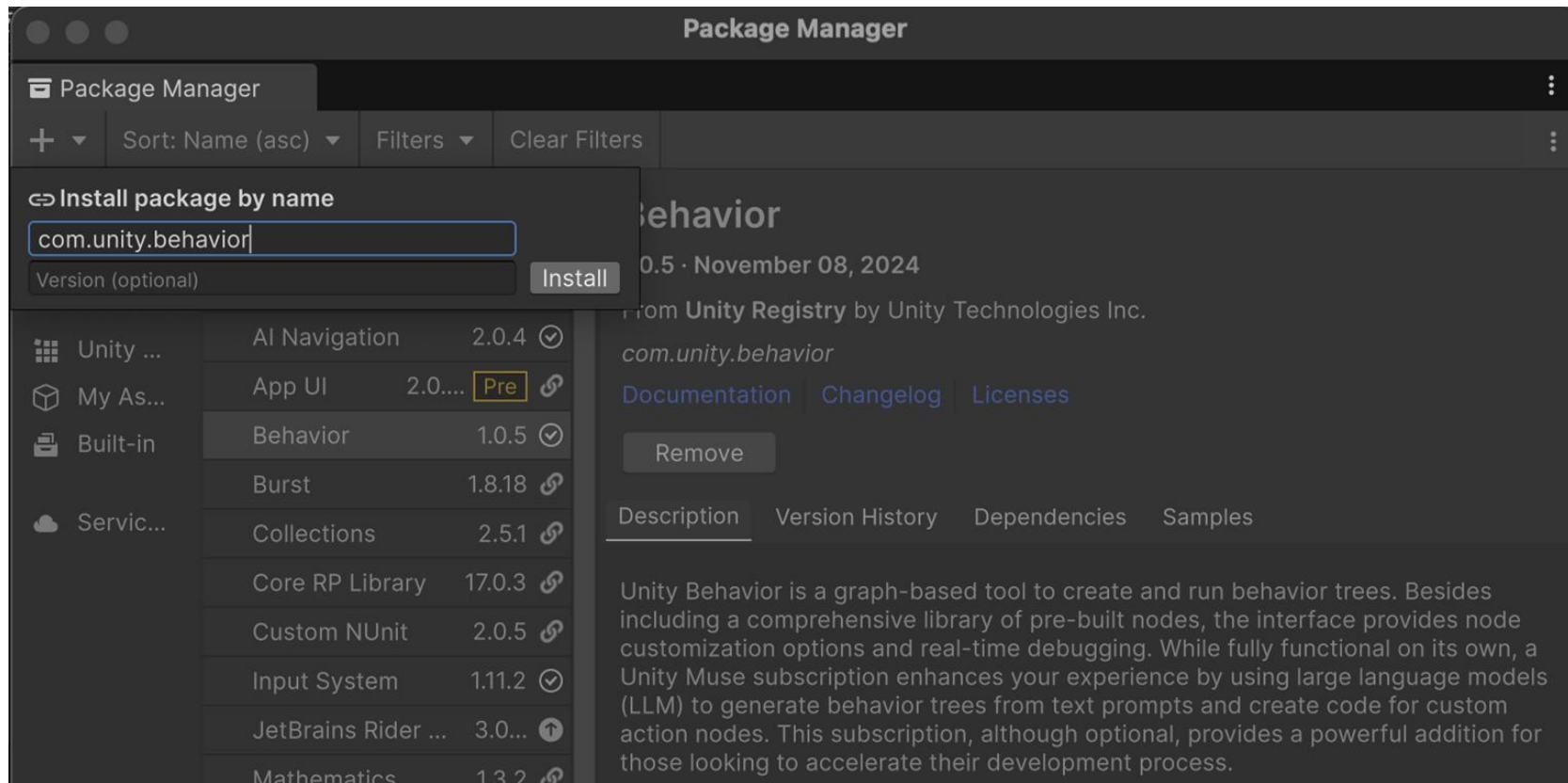
□ Ví dụ mẫu phương thức Chase();

```
45     public GameObject player; // Unchanged
46     private int chaseRange = 100;
    ? usages ? overrides ? extension methods ? exposing APIs
47     private void Chase()
48     {
49         Debug.Log(message: "Chasing the player");
50
51         // Đặt mục tiêu là người chơi
52         Agent.SetDestination(player.transform.position);
53
54         // Nếu người chơi ra khỏi phạm vi đuổi theo, chuyển về tuần tra
55         if (Vector3.Distance(a:transform.position, b:player.transform.position) > chaseRange)
56         {
57             currentState = State.Patrol;
58             // GoToNextPatrolPoint();
59         }
60     }
```

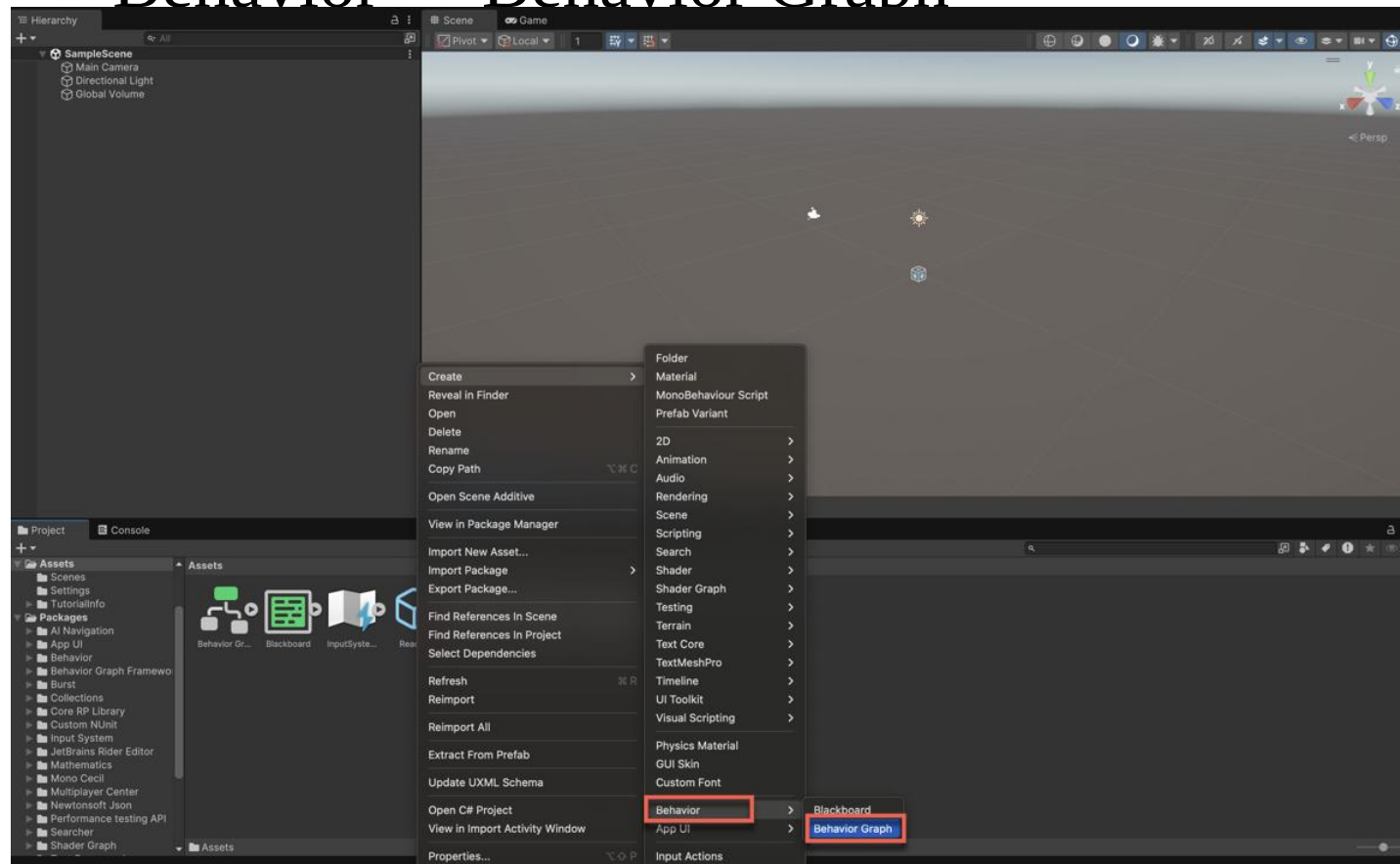
- ❑ **Behavior Trees (BT)** : là một số công cụ trực quan để tạo các hành vi nhằm điều khiển các nhân vật NPC
- ❑ BT giúp đơn giản hóa việc tạo hành vi, giúp hiện thực hóa ý tưởng một cách dễ dàng thông qua việc hỗ trợ, tương tác, dàn dựng cảnh, kích hoạt sự kiện theo ngữ cảnh, điều kiện ...

- ❑ BT là 1 công cụ không có sẵn trên Unity, để sử dụng cần cài đặt từ Asset Store hoặc Package Manager
- ❑ Có nhiều loại để lựa chọn : Behavior Designer, Playmaker hoặc [Unity Behavior](#)

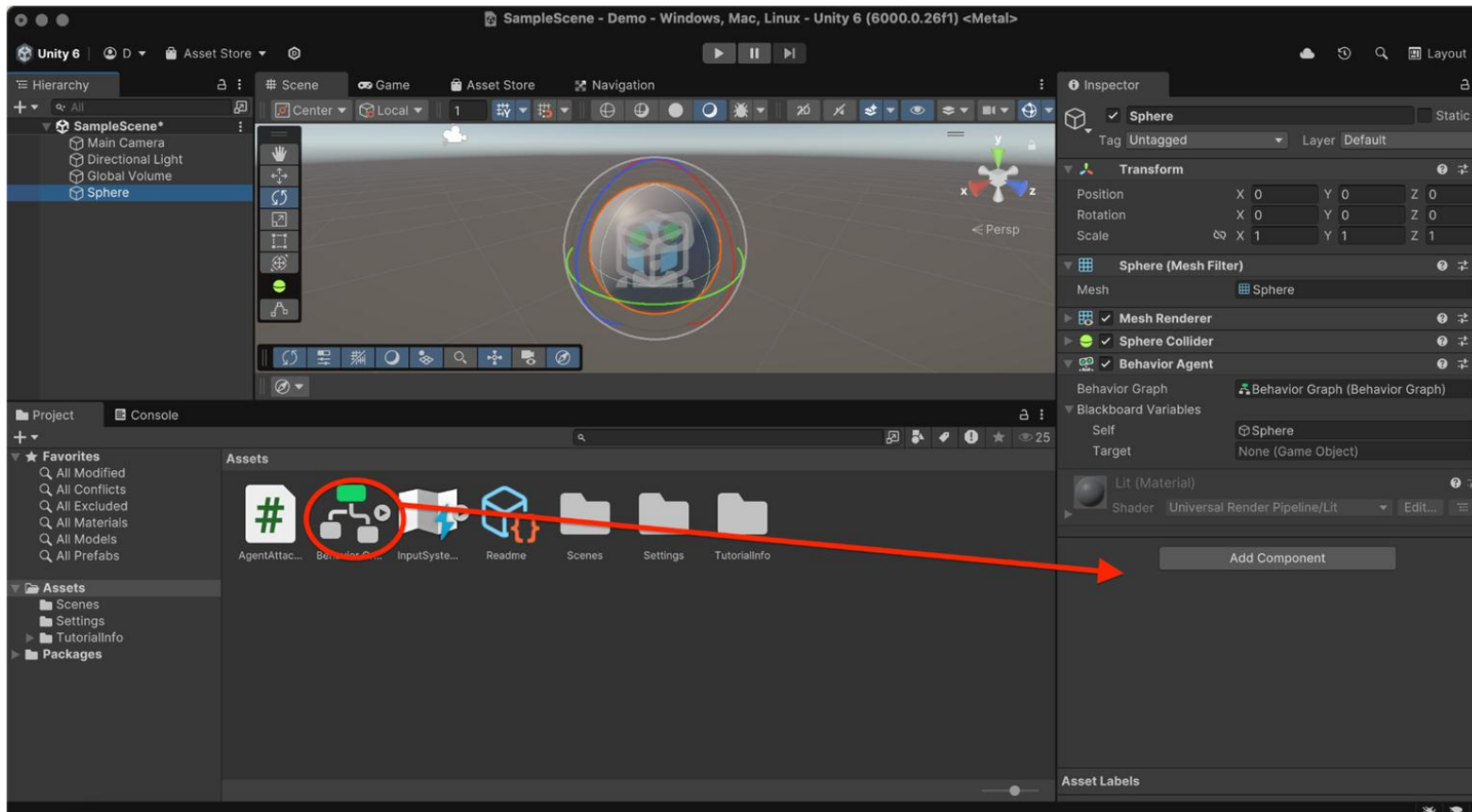
- ☐ Cách cài đặt :
- ☐ Chọn Window > Package Manager.
- ☐ Mở menu Add và chọn Add package by name....
- ☐ Nhập (hoặc sao chép và dán) com.unity.behavior vào trường Name, sau đó chọn Add.
- ☐ Chọn In Project trong Package Manager để xem Behavior được liệt kê dưới mục Packages - Unity.



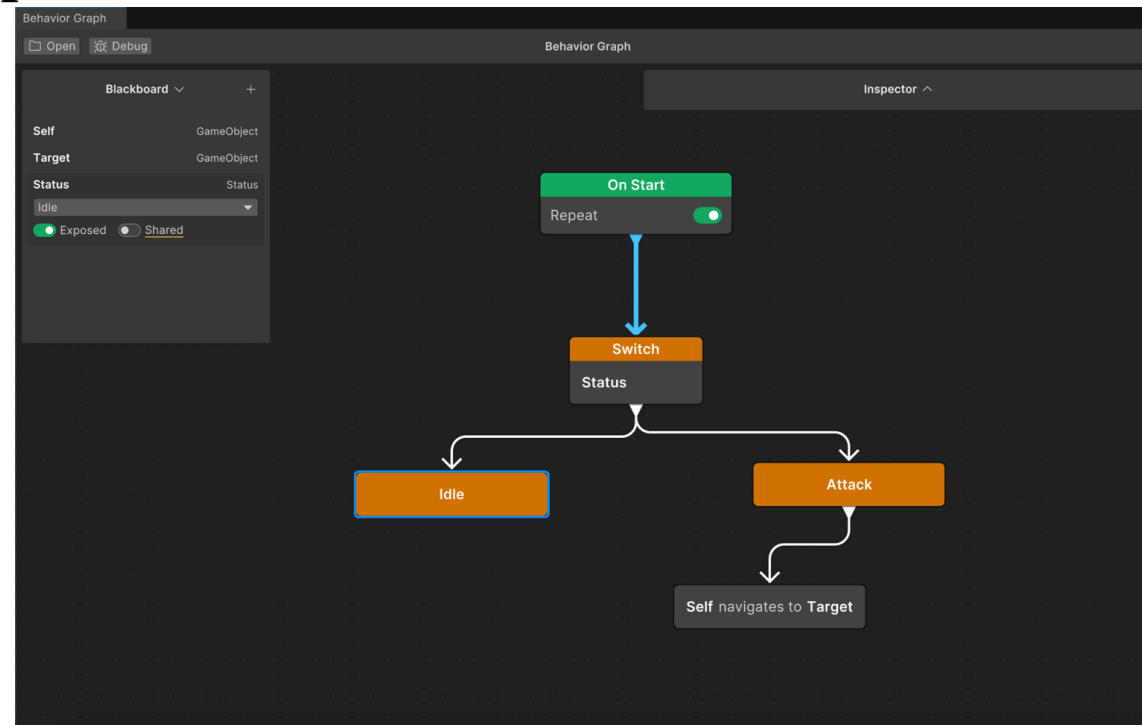
- ❑ Bấm chuột phải trên thư mục Assets chọn Create
- > Behavior -> Behavior Graph



□ Đính kèm Behavior Graph vào GameObject bằng cách kéo thả



- ❑ Mở file Behavior vừa tạo và tiến hành thêm các node điều khiển hành vi NPC thông qua Visual Graph



- ☐ Finite State Machine
- ☐ Behavior Trees
- ☐ Thực hành áp dụng FSM và BT trong Game



Kết thúc